CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

Project Report

Subject Name: Data Mining Lab

Subject Code: 24CAP-308

Project Name: STUDENT PERFORMANCE ANALYSIS SYSTEM

Submitted by:

Devansh Sood

24BCA10066

Section: 24BCA – 1 'A'

Submitted To:

Ms. Jyoti

STUDENT PERFORMANCE ANALYSIS SYSTEM

Mini Project Report — Data Preprocessing, Classification, Regression and Clustering of Student Academic Records

Project Type: Academic Lab Project / Data Mining	**Primary Language:** Python 3.x
Libraries: pandas, NumPy, scikit-learn, Matplotlib	**Development IDE:** Visual Studio Code (VS Code)
Dataset: 400 synthetic student records, 8 attributes	**Execution State:** Validated (end-to-end run)

1. Abstract

The Student Performance Analysis System applies the standard data-mining workflow to academic records in order to understand what drives student results and to identify students who may need support early. Institutions collect attendance, internal marks, previous grades and similar data every semester, but the raw records are usually incomplete and contain entry errors, so they cannot be analysed directly.

The system implements a seven-stage pipeline: dataset generation and audit, cleaning and preprocessing (missing-value imputation and outlier handling), exploratory correlation analysis, **classification** of students into Low, Medium and High performance levels with five algorithms (Decision Tree, Naive Bayes, k-NN, Logistic Regression, Random Forest), **regression** of the final score, **k-means clustering** of student profiles, and an at-risk detection step with CSV export of the results. Models are compared using a stratified train/test split and 5-fold cross-validation.

Note on data: real student records are confidential, so the project uses a synthetic dataset of 400 students that is generated with a fixed random seed and deliberately contaminated with missing values and impossible entries. The findings therefore demonstrate the method and its behaviour; they are not conclusions about real students.

2. Project Design & Implementation

The system follows a sequential data-mining pipeline in which each stage consumes the output of the previous one:

Stage	Function / Technique	Responsibility	Output
1. Data generation	generate_dataset()	Create 400 student records and inject data-quality problems	Raw CSV
2. Audit	audit()	Count missing values and impossible entries	Audit summary
3. Cleaning	clean(), encode()	Reset invalid values, median/mode imputation, encode categories	Clean CSV, target label
4. Exploration	Correlation analysis	Measure the relation of each attribute to the final score	Correlations, heatmap
5. Classification	classify()	Compare 5 classifiers; confusion matrix; tree rules; feature importance	Model table, figures

Stage	Function / Technique	Responsibility	Output
6. Regression and clustering	LinearRegression, KMeans	Predict final score; group students into profiles	R^2, MAE, cluster profiles
7. At-risk detection	Out-of-fold Random Forest	Flag students with a high probability of Low performance	at_risk_students.csv

2.1 Dataset Description

Each record describes one student. The final score (out of 100) is generated from the other attributes plus random noise, so a relationship exists for the algorithms to discover:

Attribute	Type	Range / values	Meaning
attendance_pct	Numeric	40 – 100	Percentage of classes attended
study_hours_week	Numeric	0 – 35	Self-study hours per week
prev_cgpa	Numeric	4.0 – 10.0	CGPA of the previous semester
internal_marks	Numeric	5 – 50	Internal assessment marks
sleep_hours	Numeric	3.5 – 9.5	Average sleep per night
extracurricular	Categorical	Yes / No	Takes part in extracurricular activities
parental_support	Ordinal	Low / Medium / High	Level of family academic support
final_score	Numeric (target)	0 – 100	Final exam score (regression target)
performance_level	Class (target)	Low / Medium / High	Derived: below 50, 50–69, 70 and above

2.2 Data Preprocessing

Before mining, the data is deliberately made dirty and then repaired, so that each cleaning step can be verified:

- **Missing values:** 60 cells are blank (study hours 20, sleep hours 16, internal marks 14, parental support 10).
- **Impossible values:** 6 attendance values above 100% are reset to missing, because attendance cannot exceed 100%.
- **Outliers:** 4 study-hour entries between 60 and 90 hours per week are removed using an extreme-outlier fence (Q3 + 3 × IQR).
- **Imputation:** numeric gaps are filled with the column median (robust to outliers); the categorical gap uses the mode.
- **Encoding:** extracurricular becomes 0/1 and parental support becomes an ordinal 0/1/2 so that all algorithms can use it.

2.3 Data-Mining Techniques and Evaluation

- **Classification:** predict Low / Medium / High from the seven attributes. Distance-based and linear models (k-NN, Logistic Regression) use standardised inputs. Final scores are never used as an input, to avoid target leakage.

- **Evaluation:** 80/20 stratified split for test accuracy and macro F1, plus 5-fold stratified cross-validation. Cross-validation is the more reliable figure because the test set holds only 80 students.
- **Regression:** multiple linear regression for the final score, judged by R^2 and mean absolute error (MAE).
- **Clustering:** k-means on standardised attributes; the number of clusters (2 to 6) is chosen by the silhouette score.
- **At-risk detection:** a Random Forest produces out-of-fold probabilities (each student is scored by a model that never saw that student); students with P(Low) of 0.5 or more are flagged.

3. Python Code

The complete program is in `student_performance.py`. It contains the data generator, cleaning functions, the classifier comparison and the main pipeline that produces all results and figures:

```
"""Student Performance Analysis System - Data Mining Lab (24CAP-308)

Pipeline: data generation -> audit -> cleaning -> exploration -> classification
          -> regression -> clustering -> at-risk detection -> export
"""
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, cross_val_predict, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import (accuracy_score, f1_score, confusion_matrix,
                             r2_score, mean_absolute_error, silhouette_score)

SEED = 308
LEVELS = ["Low", "Medium", "High"]


# ============================================
# 1. DATA GENERATION (synthetic student records)
# ============================================
def generate_dataset(n=400, seed=SEED):
    rng = np.random.default_rng(seed)
    attendance = np.clip(rng.normal(78, 12, n), 40, 100)
    study = np.clip(rng.normal(12, 5, n), 0, 35)
    prev_gpa = np.clip(rng.normal(7.0, 1.2, n), 4.0, 10.0)
    sleep = np.clip(rng.normal(6.8, 1.1, n), 3.5, 9.5)
    extra = rng.choice(["Yes", "No"], n, p=[0.45, 0.55])
    support = rng.choice(["Low", "Medium", "High"], n, p=[0.25, 0.45, 0.30])
    ability = (prev_gpa - 7.0) / 1.2
    internal = np.clip(28 + 5 * ability + 0.18 * (attendance - 78) + 0.25 * (study - 12)
                       + rng.normal(0, 4, n), 5, 50)
    s_eff = pd.Series(support).map({"Low": -3.0, "Medium": 0.0, "High": 3.0}).to_numpy()
```

```
    final = (58 + 5.5 * ability + 0.28 * (attendance - 78) + 0.75 * (study - 12)
             + 0.55 * (internal - 28) + s_eff - 3.0 * (sleep < 5.5)
             + rng.normal(0, 5.5, n))
    final = np.clip(final, 0, 100)
    df = pd.DataFrame({
        "attendance_pct": attendance.round(1), "study_hours_week": study.round(1),
        "prev_cgpa": prev_gpa.round(2), "internal_marks": internal.round(1),
        "sleep_hours": sleep.round(1), "extracurricular": extra,
        "parental_support": support, "final_score": final.round(1)})
    # inject realistic data-quality problems
    for col, k in [("study_hours_week", 20), ("sleep_hours", 16), ("internal_marks", 14)]:
        df.loc[rng.choice(n, k, replace=False), col] = np.nan
    df.loc[rng.choice(n, 6, replace=False), "attendance_pct"] = rng.uniform(101, 130, 6).round(1)
    df.loc[rng.choice(n, 4, replace=False), "study_hours_week"] = rng.uniform(60, 90, 4).round(1)
    df.loc[rng.choice(n, 10, replace=False), "parental_support"] = np.nan
    return df


def level_of(score):
    return "Low" if score < 50 else ("Medium" if score < 70 else "High")


# ========================================
# 2. DATA AUDIT AND CLEANING
# ========================================
def audit(df):
    print(f"Records: {len(df)} | Attributes: {df.shape[1]}")
    miss = df.isna().sum()
    print("Missing values:", {k: int(v) for k, v in miss.items() if v})
    print(f"Attendance > 100%: {(df.attendance_pct > 100).sum()} | "
          f"Study hours > 50: {(df.study_hours_week > 50).sum()}")


def clean(df):
    df = df.copy()
    log = {}
    bad = df.attendance_pct > 100
    log["attendance_fixed"] = int(bad.sum())
    df.loc[bad, "attendance_pct"] = np.nan            # impossible value -> missing
    q1, q3 = df.study_hours_week.quantile([0.25, 0.75])
    hi = q3 + 3 * (q3 - q1)                           # extreme-outlier fence
    out = df.study_hours_week > hi
    log["study_outliers"] = int(out.sum())
    df.loc[out, "study_hours_week"] = np.nan
    log["values_imputed"] = int(df.isna().sum().sum())
    for c in ["attendance_pct", "study_hours_week", "internal_marks", "sleep_hours"]:
        df[c] = df[c].fillna(df[c].median())          # median imputation
    df["parental_support"] = df["parental_support"].fillna(df["parental_support"].mode()[0])
    df["performance_level"] = df.final_score.apply(level_of)
    return df, log


def encode(df):
    X = df.drop(columns=["final_score", "performance_level"]).copy()
    X["extracurricular"] = (X["extracurricular"] == "Yes").astype(int)
    X["parental_support"] = X["parental_support"].map({"Low": 0, "Medium": 1, "High": 2})
    return X


# ========================================
# 3. CLASSIFICATION
# ========================================
def classify(X, y):
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
    models = {
        "Decision Tree": DecisionTreeClassifier(max_depth=4, random_state=SEED),
        "Naive Bayes": GaussianNB(),
        "k-NN (k=7)": make_pipeline(StandardScaler(), KNeighborsClassifier(7)),
        "Logistic Regression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
        "Random Forest": RandomForestClassifier(200, random_state=SEED),
    }
    cv = StratifiedKFold(5, shuffle=True, random_state=SEED)
    rows = []
    for name, m in models.items():
        cvs = cross_val_score(m, X, y, cv=cv)
        m.fit(Xtr, ytr)
        pred = m.predict(Xte)
        rows.append({"Model": name, "CV acc": cvs.mean(), "CV std": cvs.std(),
                     "Test acc": accuracy_score(yte, pred),
                     "Macro F1": f1_score(yte, pred, average="macro")})
    res = pd.DataFrame(rows).sort_values("CV acc", ascending=False).reset_index(drop=True)
    return res, models, (Xtr, Xte, ytr, yte)
```

```
# ========================================
# 4. MAIN PIPELINE
# ========================================
def main():
    line = "=" * 78
    print(line + "\n>>> STUDENT PERFORMANCE ANALYSIS SYSTEM (DATA MINING LAB) <<<\n" + line)

    print("[1/7] Generating and auditing student dataset...")
    raw = generate_dataset()
    raw.to_csv("student_data_raw.csv", index=False)
    audit(raw)

    print("\n[2/7] Cleaning and preprocessing...")
    df, log = clean(raw)
    df.to_csv("student_data_clean.csv", index=False)
    print(f"  Impossible attendance values reset: {log['attendance_fixed']} | "
          f"Extreme study-hour outliers reset: {log['study_outliers']}")
    print(f"  Missing cells imputed (median / mode): {log['values_imputed']} | "
          f"Remaining missing: {int(df.isna().sum().sum())}")
    print("  Performance levels:", df.performance_level.value_counts().reindex(LEVELS).to_dict())

    print("\n[3/7] Exploratory analysis (correlation with final score)...")
    X = encode(df)
    corr = X.assign(final_score=df.final_score).corr()["final_score"].drop("final_score")
    for k, v in corr.sort_values(ascending=False).items():
        print(f"  {k:<18} r = {v:+.3f}")

    print("\n[4/7] Classification: predicting performance level...")
    y = df.performance_level
    res, models, (Xtr, Xte, ytr, yte) = classify(X, y)
    print(f"  {'Model':<20}{'CV acc':>8}{'CV std':>8}{'Test acc':>10}{'Macro F1':>10}")
    for _, r in res.iterrows():
        print(f"  {r['Model']:<20}{r['CV acc']:>8.3f}{r['CV std']:>8.3f}{r['Test acc']:>10.3f}{r['Macro F1']:>10.3f}")
    best = res.loc[0, "Model"]
    cm = confusion_matrix(yte, models[best].predict(Xte), labels=LEVELS)
    print(f"\n  Confusion matrix of best model ({best}); rows = actual, cols = predicted")
    print("  " + " " * 8 + "".join(f"{l:>8}" for l in LEVELS))
    for l, row in zip(LEVELS, cm):
        print(f"  {l:<8}" + "".join(f"{v:>8}" for v in row))
    tree = DecisionTreeClassifier(max_depth=3, random_state=SEED).fit(X, y)
    print("\n  Decision-tree rules (depth 3):")
    for ln in export_text(tree, feature_names=list(X.columns)).splitlines():
```

```
        print("   " + ln)
    rf = models["Random Forest"]
    imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
    print("\n  Random Forest feature importance:")
    for k, v in imp.items():
        print(f"   {k:<18}{v:.3f}")

    print("\n[5/7] Regression: predicting final score...")
    Xa, Xb, ya, yb = train_test_split(X, df.final_score, test_size=0.2, random_state=SEED)
    lr = LinearRegression().fit(Xa, ya)
    pr = lr.predict(Xb)
    print(f"  R2 = {r2_score(yb, pr):.3f} | MAE = {mean_absolute_error(yb, pr):.2f} marks")
    print("  Coefficients (marks per unit):")
    for c, v in zip(X.columns, lr.coef_):
        print(f"   {c:<18}{v:+.2f}")

    print("\n[6/7] Clustering students (k-means)...")
    Z = StandardScaler().fit_transform(X)
    sil = {}
    for k in range(2, 7):
        sil[k] = silhouette_score(Z, KMeans(k, n_init=10, random_state=SEED).fit_predict(Z))
    print("  Silhouette by k: " + ", ".join(f"k={k}: {v:.3f}" for k, v in sil.items()))
    k = max(sil, key=sil.get)
    print(f"  Best k by silhouette: {k}")
    labels = KMeans(k, n_init=10, random_state=SEED).fit_predict(Z)
    df["cluster"] = labels
    prof = df.groupby("cluster")[["attendance_pct", "study_hours_week", "prev_cgpa",
                                  "internal_marks", "final_score"]].mean().round(1)
    prof.insert(0, "students", df.groupby("cluster").size())
    print(f"  Cluster profiles (k = {k}):")
    print("  " + prof.to_string().replace("\n", "\n  "))

    print("\n[7/7] At-risk detection and export...")
    cvk = StratifiedKFold(5, shuffle=True, random_state=SEED)
    oof = cross_val_predict(RandomForestClassifier(200, random_state=SEED), X, y, cv=cvk, method="predict_proba")
    classes = sorted(y.unique())                       # column order of predict_proba
    proba = pd.Series(oof[:, classes.index("Low")], index=df.index)
    risk = df[(proba >= 0.5)]
    print(f"  Students flagged at-risk (P(Low) >= 0.5): {len(risk)} of {len(df)}")
    print(f"  Mean attendance of flagged: {risk.attendance_pct.mean():.1f}% "
          f"vs others: {df.drop(risk.index).attendance_pct.mean():.1f}%")
    risk.assign(p_low=proba[risk.index].round(2)).to_csv("at_risk_students.csv", index=False)
    df.to_csv("student_data_clean.csv", index=False)

    # ---------- figures ----------
    plt.figure(figsize=(6, 4.6))
    c = X.assign(final_score=df.final_score).corr()
    plt.imshow(c, cmap="RdBu_r", vmin=-1, vmax=1)
    plt.xticks(range(len(c)), c.columns, rotation=45, ha="right", fontsize=7)
    plt.yticks(range(len(c)), c.columns, fontsize=7)
    plt.colorbar(shrink=0.8)
    plt.title("Correlation matrix", fontsize=10)
    plt.tight_layout(); plt.savefig("fig_correlation.png", dpi=170); plt.close()

    plt.figure(figsize=(6, 4.2))
    plt.barh(res["Model"][::-1], res["CV acc"][::-1], xerr=res["CV std"][::-1], color="#1F3A8A")
    plt.xlim(0.5, 1.0); plt.xlabel("5-fold cross-validated accuracy")
    plt.title("Classifier comparison", fontsize=10)
    plt.tight_layout(); plt.savefig("fig_models.png", dpi=170); plt.close()

    p2 = PCA(2, random_state=SEED).fit_transform(Z)
    plt.figure(figsize=(6, 4.2))
    for cl, col in zip(range(k), ["#C0392B", "#1F3A8A", "#0E8F86", "#E69F00", "#7F8C8D", "#8E44AD"]):
        plt.scatter(p2[labels == cl, 0], p2[labels == cl, 1], s=14, c=col, label=f"Cluster {cl}")
    plt.xlabel("PC1"); plt.ylabel("PC2"); plt.legend(fontsize=8)
    plt.title("Student clusters (PCA projection)", fontsize=10)
    plt.tight_layout(); plt.savefig("fig_clusters.png", dpi=170); plt.close()

    plt.figure(figsize=(6, 4.2))
    plt.barh(imp.index[::-1], imp.values[::-1], color="#0E8F86")
    plt.xlabel("Importance"); plt.title("Random Forest feature importance", fontsize=10)
    plt.tight_layout(); plt.savefig("fig_importance.png", dpi=170); plt.close()

    print("\n[EXPORT COMPLETE] student_data_clean.csv, at_risk_students.csv and 4 figures saved.")
    print(line)


if __name__ == "__main__":
    main()
```

4. Output / Screenshots

The program was executed with `python student_performance.py`. All seven stages completed and printed the following results (the random seed is fixed, so the output is identical on every run):

```
================================================================================
>>> STUDENT PERFORMANCE ANALYSIS SYSTEM (DATA MINING LAB) <<<
================================================================================
[1/7] Generating and auditing student dataset...
Records: 400 | Attributes: 8
Missing values: {'study_hours_week': 20, 'internal_marks': 14, 'sleep_hours': 16, 'parental_support': 10}
Attendance > 100%: 6 | Study hours > 50: 4

[2/7] Cleaning and preprocessing...
  Impossible attendance values reset: 6 | Extreme study-hour outliers reset: 4
  Missing cells imputed (median / mode): 70 | Remaining missing: 0
  Performance levels: {'Low': 93, 'Medium': 237, 'High': 70}

[3/7] Exploratory analysis (correlation with final score)...
  internal_marks     r = +0.732
  prev_cgpa          r = +0.645
  study_hours_week   r = +0.397
  attendance_pct     r = +0.251
  parental_support   r = +0.118
  sleep_hours        r = +0.104
  extracurricular    r = +0.051

[4/7] Classification: predicting performance level...
  Model                 CV acc  CV std  Test acc  Macro F1
  Logistic Regression    0.750   0.051     0.775     0.755
  Naive Bayes            0.735   0.040     0.738     0.703
  Random Forest          0.700   0.035     0.738     0.689
  k-NN (k=7)             0.685   0.034     0.713     0.637
  Decision Tree          0.673   0.048     0.675     0.605

  Confusion matrix of best model (Logistic Regression); rows = actual, cols = predicted
              Low  Medium    High
  Low          13       6       0
  Medium        7      40       0
  High          0       5       9

  Decision-tree rules (depth 3):
   |--- internal_marks <= 23.25
   |   |--- study_hours_week <= 10.10
   |   |   |--- internal_marks <= 21.05
   |   |   |   |--- class: Low
   |   |   |--- internal_marks >  21.05
   |   |   |   |--- class: Low
   |   |--- study_hours_week >  10.10
```

```
    |   |   |--- internal_marks <= 17.20
    |   |   |   |--- class: Low
    |   |   |--- internal_marks >  17.20
    |   |   |   |--- class: Medium
    |--- internal_marks >  23.25
    |   |--- internal_marks <= 33.95
    |   |   |--- prev_cgpa <= 8.56
    |   |   |   |--- class: Medium
    |   |   |--- prev_cgpa >  8.56
    |   |   |   |--- class: High
    |   |--- internal_marks >  33.95
    |   |   |--- prev_cgpa <= 6.94
    |   |   |   |--- class: Medium
    |   |   |--- prev_cgpa >  6.94
    |   |   |   |--- class: High

  Random Forest feature importance:
   internal_marks    0.304
   prev_cgpa         0.213
   study_hours_week  0.174
   attendance_pct    0.133
   sleep_hours       0.106
   parental_support  0.041
   extracurricular   0.029

[5/7] Regression: predicting final score...
  R2 = 0.783 | MAE = 4.29 marks
  Coefficients (marks per unit):
   attendance_pct    +0.24
   study_hours_week  +0.87
   prev_cgpa         +4.43
   internal_marks    +0.42
   sleep_hours       +0.48
   extracurricular   +0.64
   parental_support  +2.49

[6/7] Clustering students (k-means)...
  Silhouette by k: k=2: 0.148, k=3: 0.144, k=4: 0.140, k=5: 0.139, k=6: 0.133
  Best k by silhouette: 2
  Cluster profiles (k = 2):
           students  attendance_pct  study_hours_week  prev_cgpa  internal_marks  final_score
  cluster
  0             185           81.4              12.5        7.9            33.4         65.3
  1             215           76.0              11.3        6.3            23.5         51.6

[7/7] At-risk detection and export...
  Students flagged at-risk (P(Low) >= 0.5): 69 of 400
  Mean attendance of flagged: 74.6% vs others: 79.3%

[EXPORT COMPLETE] student_data_clean.csv, at_risk_students.csv and 4 figures saved.
================================================================================
```

4.1 Visual Results

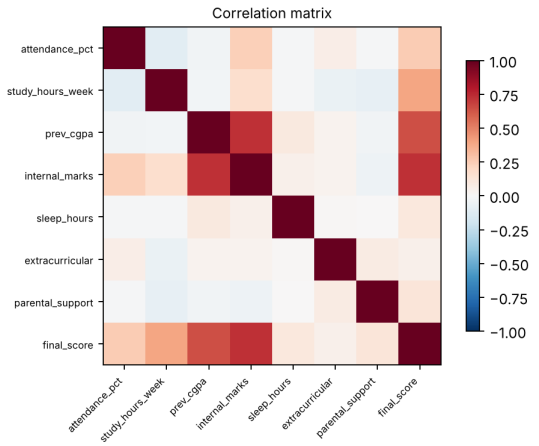


Figure 1. Correlation matrix of all attributes with the final score

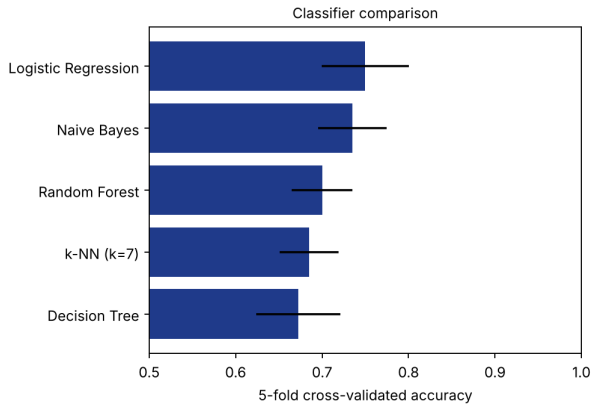


Figure 2. Cross-validated accuracy of the five classifiers (error bar = 1 std. dev.)

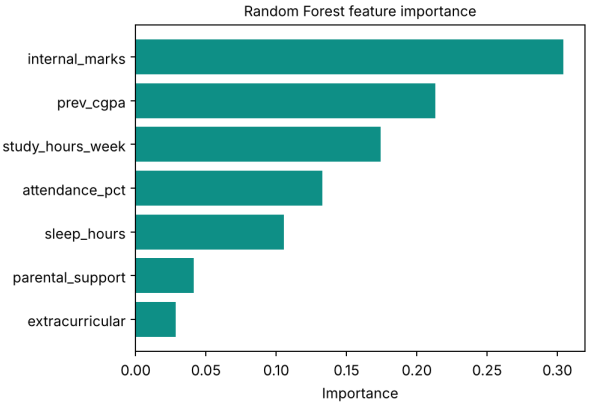


Figure 3. Random Forest feature importance

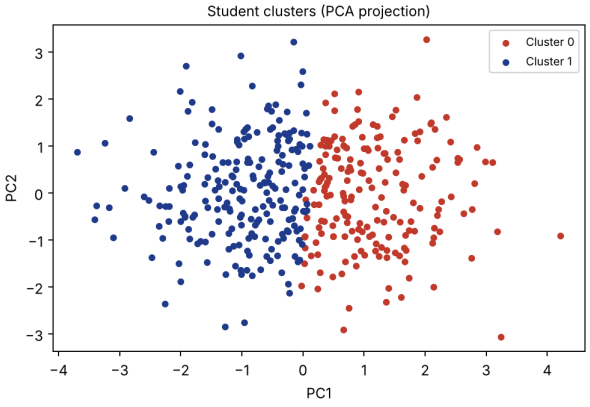


Figure 4. Two student clusters shown on the first two principal components

4.2 Results and Interpretation

- **Cleaning:** all 70 problem cells (60 missing + 10 invalid) were repaired and none remain. The performance levels are Low 93 (23%), Medium 237 (59%) and High 70 (18%), a moderately imbalanced but usable distribution.
- **Relationships:** the final score correlates most strongly with internal marks (r = +0.73) and previous CGPA (r = +0.65), then study hours (+0.40) and attendance (+0.25). Extracurricular activity is almost uncorrelated (+0.05).
- **Classification:** Logistic Regression was best with 75.0% cross-validated accuracy (± 5.1) and 77.5% test accuracy (macro F1 0.755). Naive Bayes (73.5%) is within one standard deviation of it, so the two cannot be separated reliably. The Decision Tree was weakest (67.3%). A linear model wins here because the scores were generated by a roughly linear process; on real data a tree ensemble could do better.
- **Error pattern:** of 80 test students, 62 were classified correctly. Every mistake was between neighbouring levels; no Low student was predicted High and no High student was predicted Low. Medium was recognised best (40 of 47), High and Low less well (9 of 14 and 13 of 19).
- **Feature importance:** internal marks (0.30), previous CGPA (0.21), study hours (0.17) and attendance (0.13) dominate; parental support (0.04) and extracurricular activity (0.03) contribute little. The tree rules show the same order: low internal marks combined with few study hours leads to Low.
- **Regression:** R^2 = 0.783 and MAE = 4.29 marks, so the model explains about 78% of the variation in final score. Coefficients are close to the values used to generate the data (for example +4.43 marks per CGPA point against 4.58 planted, and +2.49 marks per support level against 3.0 planted), which confirms the pipeline recovers real structure.
- **Clustering:** the silhouette score was highest at k = 2 (0.148) but is low overall, so students form a continuum rather than distinct groups. The two clusters separate stronger students (185 students: CGPA 7.9, internal marks 33.4, mean final score 65.3) from those needing support (215 students: CGPA 6.3, internal marks 23.5, final score 51.6).
- **At-risk detection:** 69 students were flagged; 45 of them (65%) really were Low performers, and these 45 are 48% of all 93 Low students. Lowering the 0.5 threshold would find more at-risk students at the cost of more false alarms.

4.3 Exported Files

- **student_data_raw.csv** and **student_data_clean.csv**: the dataset before and after cleaning (the clean file also holds the performance level and cluster of each student).
- **at_risk_students.csv**: the 69 flagged students with their risk probability.
- **fig_correlation.png, fig_models.png, fig_importance.png, fig_clusters.png**: the four figures above.

5. Limitations

- The data is synthetic and its relationships were built in, so accuracy figures will differ on real records; the experiment shows that the pipeline works, not how real students behave.
- The test set has only 80 students, so test accuracy varies by several percentage points; differences between the top models are inside the cross-validation spread.
- Median imputation weakens relationships slightly (the internal-marks coefficient is 0.42 against 0.55 planted).
- Performance levels depend on fixed cut-offs (50 and 70); students near a boundary are inherently hard to classify.
- Predictions about students should support, not replace, a teacher's judgement, and should not be used to penalise anyone.

6. Conclusion

The Student Performance Analysis System demonstrates a complete data-mining workflow: cleaning messy records, exploring them, comparing five classifiers, predicting scores with regression, discovering student profiles by clustering and flagging at-risk students. Internal marks, previous CGPA, study hours and attendance emerged as the main drivers of performance, and a simple, interpretable Logistic Regression model gave the best balance of accuracy and transparency. Future work includes applying the same pipeline to a real dataset such as the public UCI Student Performance data, tuning the models, tuning the at-risk threshold, and adding association-rule mining.

7. References

[1] Cortez, P., & Silva, A. (2008). Using data mining to predict secondary school student performance. *Proceedings of the 5th FUture BUsiness TEChnology Conference (FUBUTEC)*, 5–12.

[2] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[3] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[4] Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

8. GitHub Repository Link

Project Repository & Source Control

GitHub Repository URL: https://github.com/devansh349/Student-Performance-Analysis-System